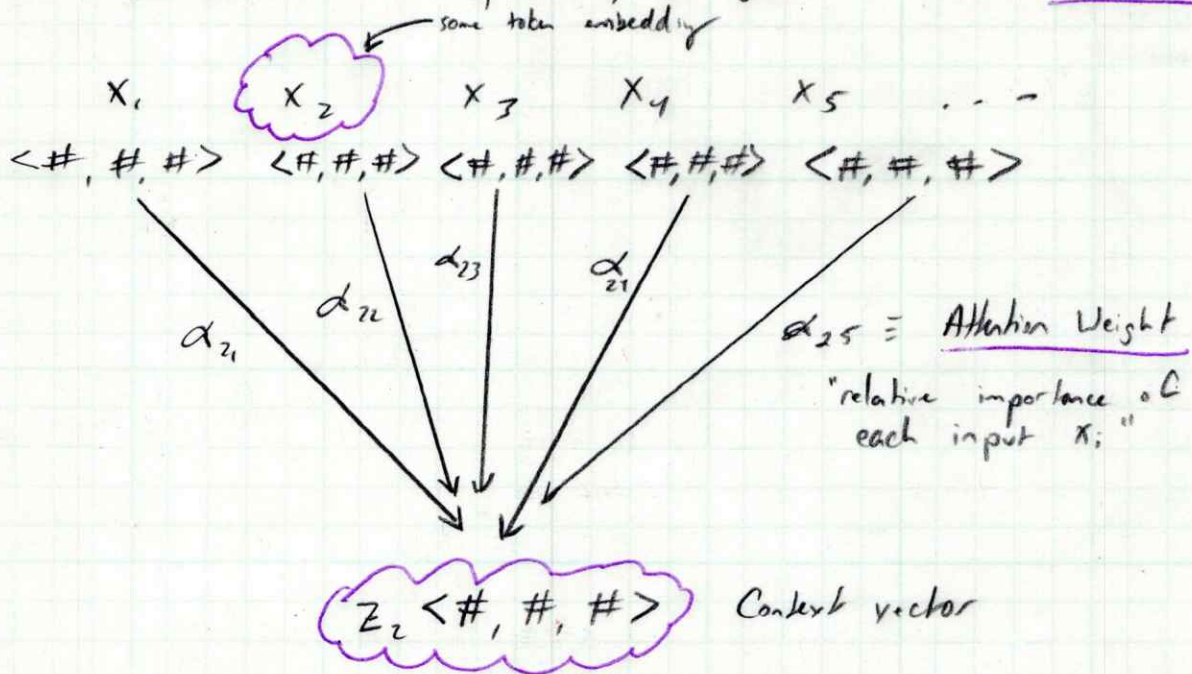


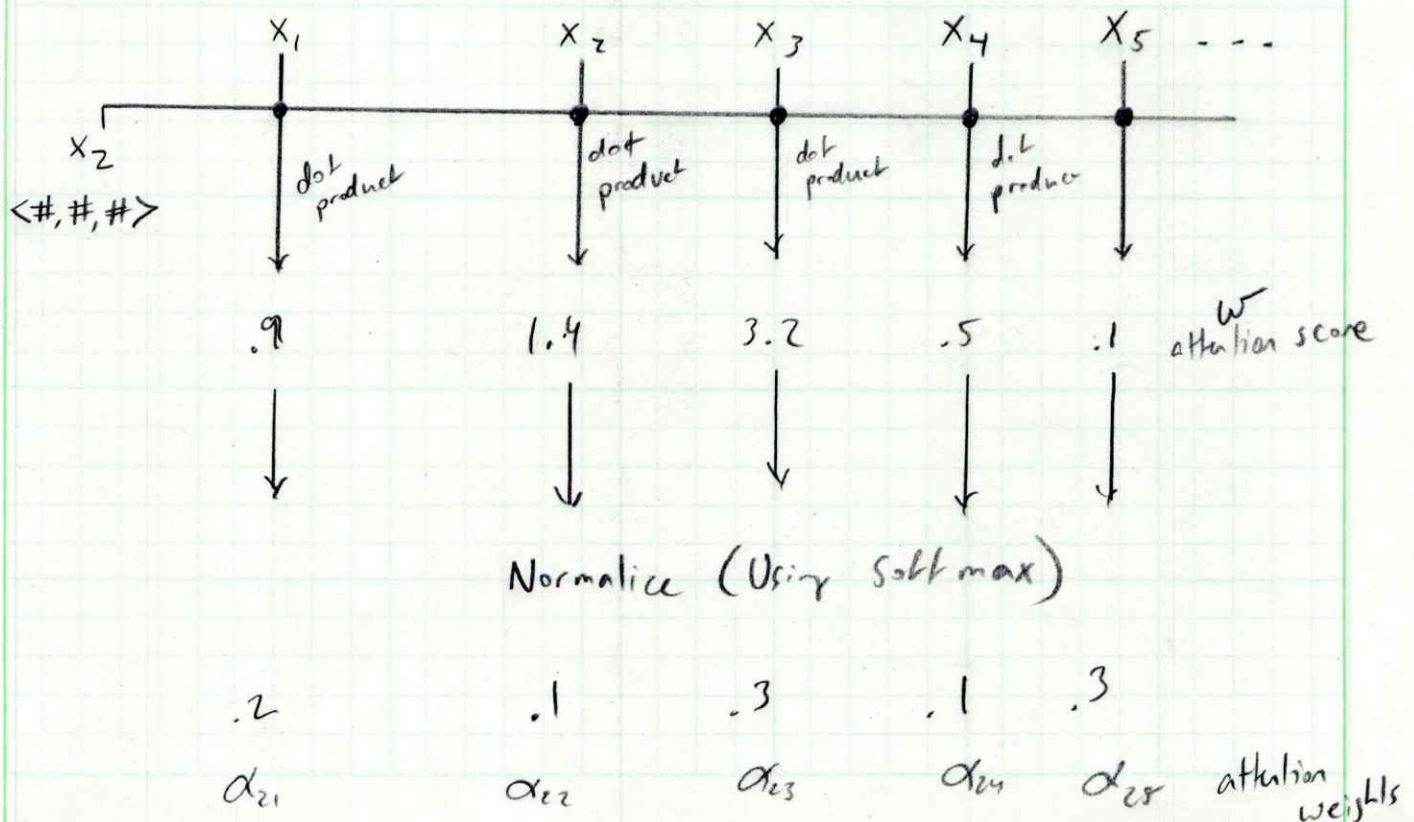
Simplified Self-Attention

Each ELEMENT in the input sequence gets its own CONTEXT VECTOR

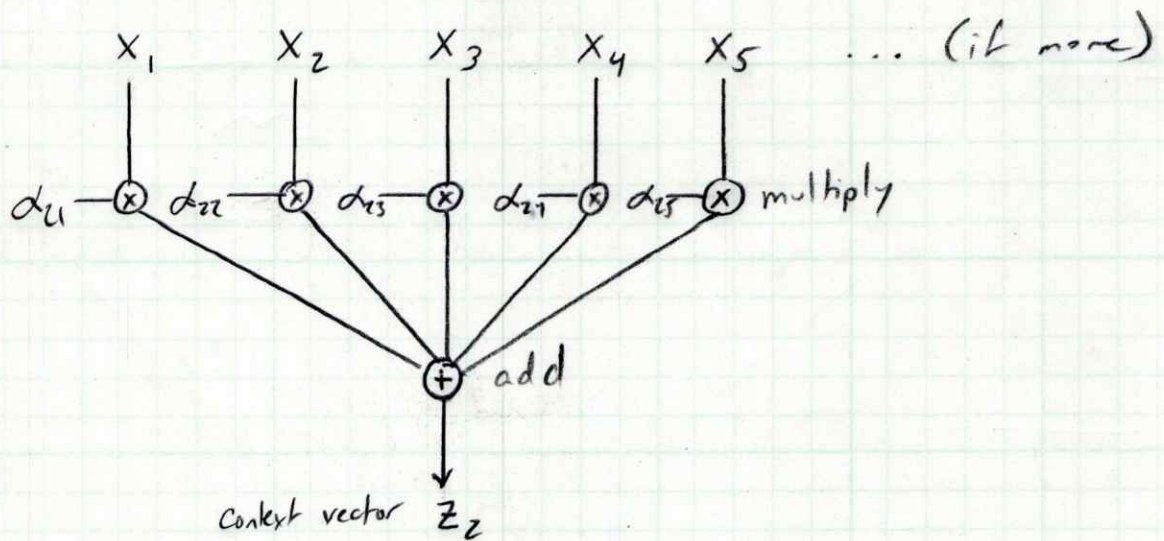


In this example X_2 is my QUERY

To get the CONTEXT VECTOR, first we need the ATTENTION SCORE (w)



Now we calculate the Context Vector



Now repeat to calc All Attention Weights for All tokens

	x_1	x_2	x_3	x_4	x_5
x_1	#	#	#	#	#
x_2	#	#	#	#	#
x_3	#	#	#	#	#
x_4	#	#	#	#	#
x_5	#	#	#	#	#

The attention weights for the second token (x_2) are highlighted in the original image:

	x_1	x_2	x_3	x_4	x_5
x_1	#	#	#	#	#
x_2	#	α_{21}	α_{22}	α_{23}	α_{24}
x_3	#	#	#	#	#
x_4	#	#	#	#	#
x_5	#	#	#	#	#

attention_scores =

dot(inputs, inputs transpose)

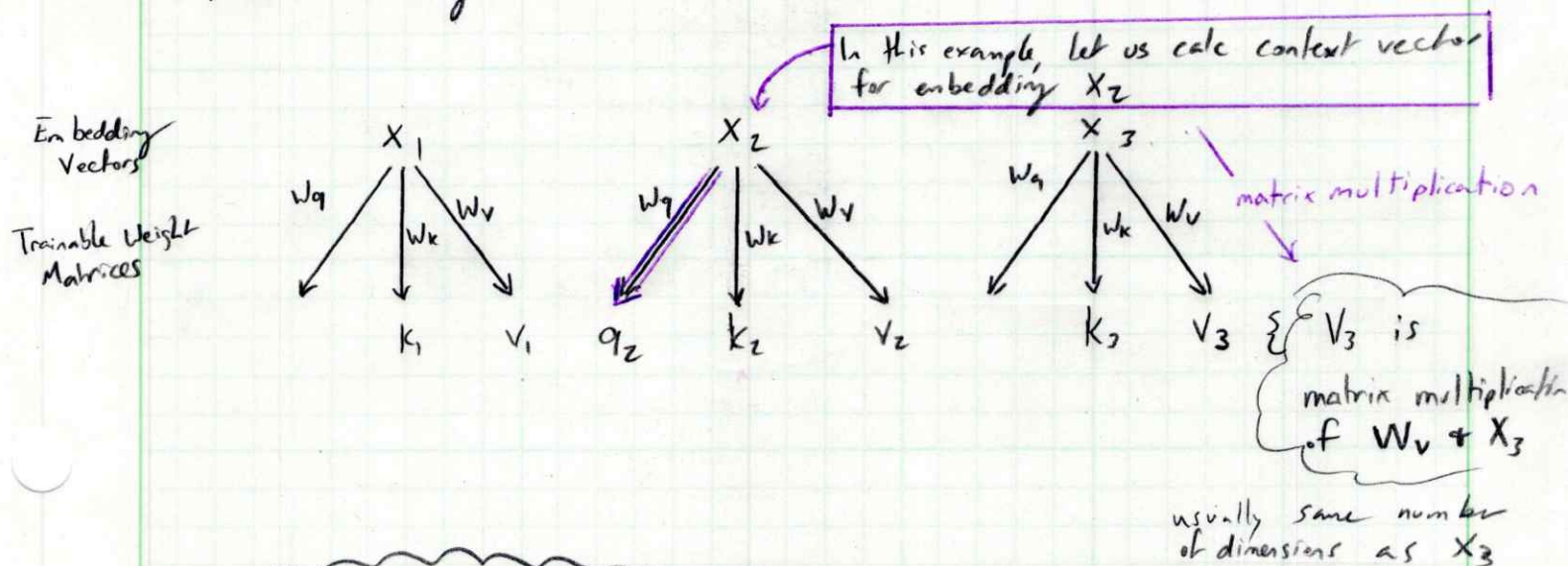
Calc all context vectors z_1, z_2, z_3, z_4, z_5

Self-Attention with Trainable Weights

aka scaled dot product attention

approach is the same $\rightarrow$ we want to build (calculate) context vectors
over input vectors specific to a given input.

- We add to the mix WEIGHT MATRICES that are updated during model training

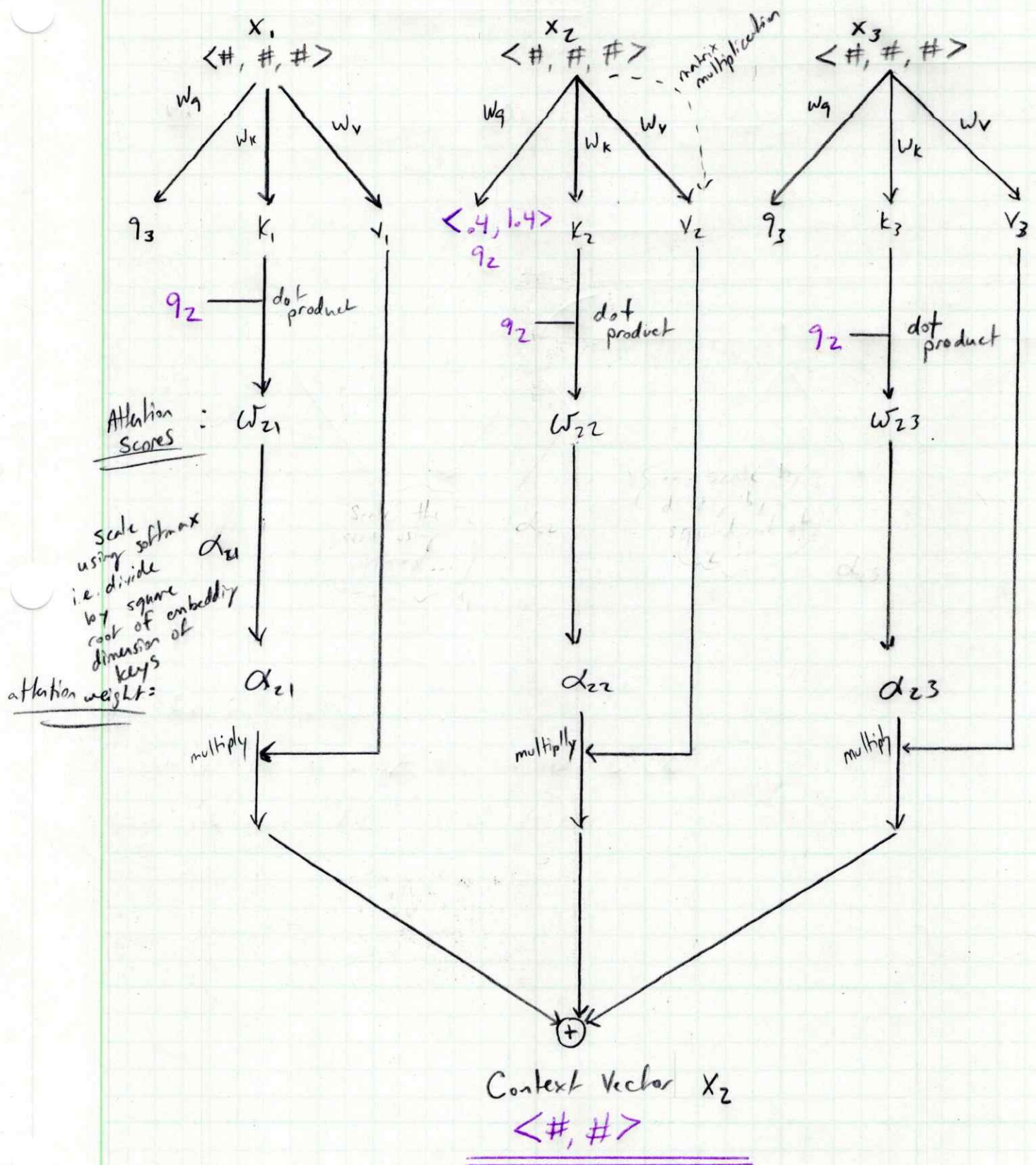


Oh by the way!!!

W weight parameters $\neq$ Attention Weights α
 (learned coefficients) (Context)

attention weights determine the extent to which a context vector depends on the different parts of the input

Now we calculate attention scores w



Context vector is a weighted sum over the value vectors.
Attention weights (α) provide meaning of how important that embedding is

"Query, Key, Value" → borrowed from Database Theory

Query - I'm probing the input sequence for a specific embedding

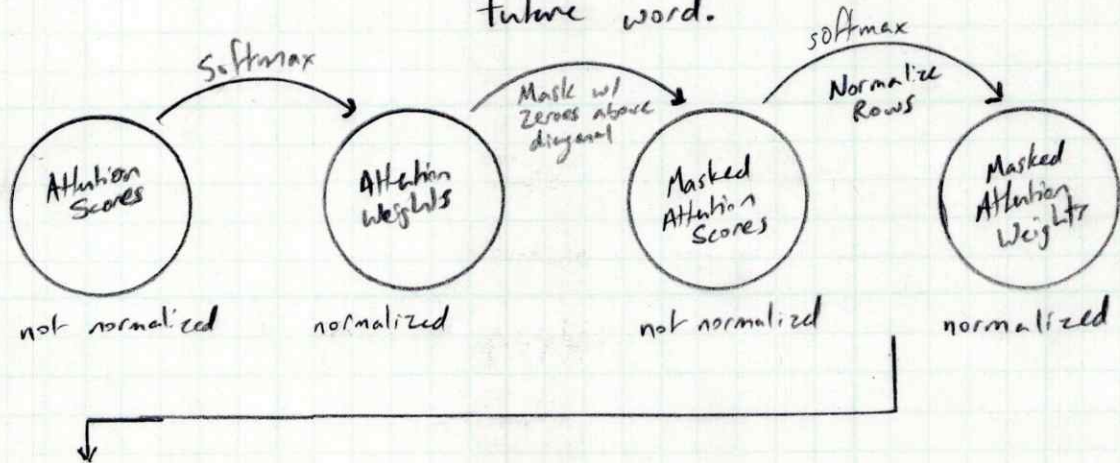
Key - In the attention mechanism, each item in the input sequence has an associated key

Value - Once the model determines which keys (and thus which parts of the input) are the most relevant to the query, it retrieves the corresponding values.

Casual Attention aka "Masked Attention"

	x_1	x_2	x_3	x_4	x_5
x_1	#	#	#	#	#
x_2	#	#	#	#	#
x_3	#	#	#	#	#
x_4	#	#	#	#	#
x_5	#	#	#	#	#

~ These are the attention weights.
Remember! We need to mask what's in purple so the model only considers what's come before to predict the future word.



softmax converts inputs into a probability function
so we use a mask of $-\infty$:

$-\infty$	$-\infty$	$-\infty$		
	$-\infty$	$-\infty$		
		$-\infty$		
			$-\infty$	
				$-\infty$

But we also apply a dropout rate to prevent overtraining the model

Applied:

(a) after calculating attention weights and/or

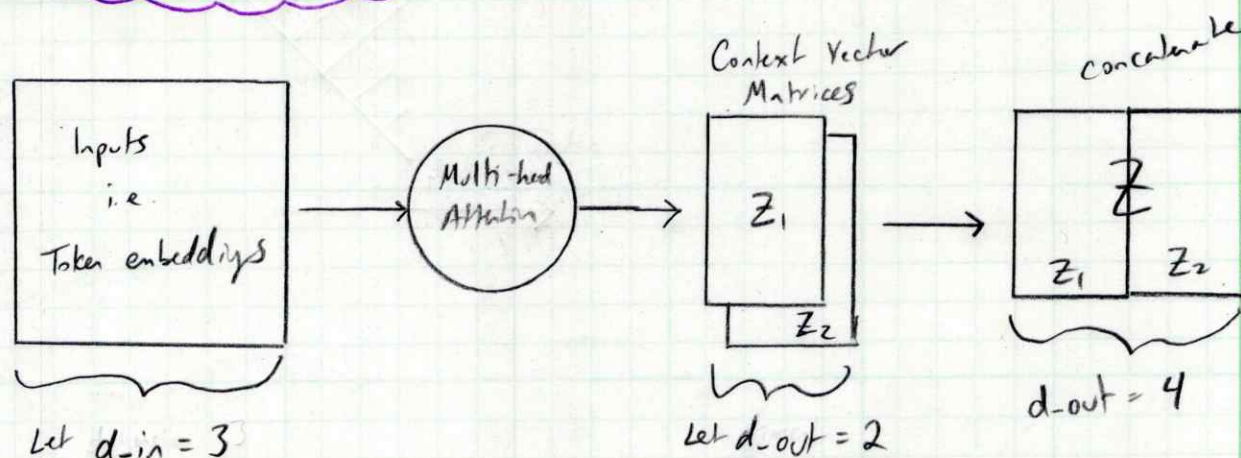
(b) after applying attention weights to value vectors

	x_1	x_2	x_3	x_4	x_5
x_1	0	0	0	0	0
x_2	0	0	0	0	0
x_3	0	0	0	0	0
x_4	0	0	0	0	0
x_5	0	0	0	0	0

← Typical rate around 50%

To compensate for the reduction in active elements, the values of the remaining elements in the matrix are scaled up by a factor of $\frac{1}{0.5} = 2$

Multi Head Attention



"The main idea behind multi-head attention is to run the attention mechanism multiple times (in parallel) with different learned linear projections."

↑ I think the author is referring to W_Q , W_K & W_V

The book didn't mention this, but a follow up question with ChatGPT helped crystallize it. The different heads invariably end up "learning" different things because of -

- (a) different weight initializations
- (b) nonlinear learning dynamics
- (c) shared downstream loss across heads
- (d) limited dimensionality of each head.

You are basically rolling balls downhill and they end up in different spots based on the embeddings.